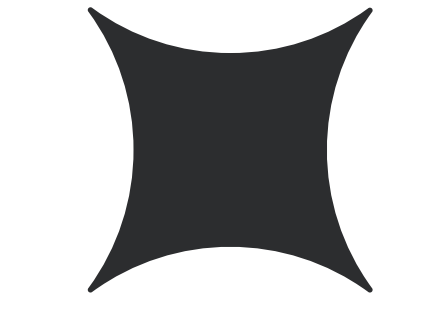




HENNGE

CHALLENGE DETAILS

Introduction

Thank you for your application to HENNGE.

This challenge is intended for **backend position** or **global internship (full-stack)** applicants.

This challenge consists of three missions. After successfully completing Mission 3, you will receive an email with a link to submit your CV and Cover Letter (CL). Once we receive them, our engineers will review your code and documents (CV & CL) together, and we will contact you via email with the screening result. If you haven't received your result within a month, please let us know!

Tips: Due to the competitiveness of our application process, we highly recommend applicants to explain their motivation clearly in their Cover Letter (CL).

Mission 1: Write a program which fulfills the requirements below

Description

- The input will be a list of integers, each separated by a newline character.
- The first line of the input will be an integer N ($1 \leq N \leq 100$), indicating the number of test cases to follow.
- Each of the test cases will consist of a line with an integer X ($0 < X \leq 100$), followed by another line consisting of X number of space-separated integers Y_n ($-100 \leq Y_n \leq 100$).
- For each test case, calculate the power of four of Y_n , excluding when Y_n is positive, and print the calculated sum in the output.
- Note:** There should be no output until all the input has been received.
- Note 2:** Do not put blank lines between test cases solutions.
- Note 3:** Take input from standard input, and output to standard output.
- Note 4:** The final output is guaranteed to be within the `int32` range.
- Note 5:** It is possible that X and the number of integers Y_n may not be equal. If that is the case, print `-1` as the output.

Rules

Write your solution using [Go Programming Language](#) or [Python Programming Language](#). **Do not** submit your solution with both languages at once!

You may only use standard library packages. In addition, an extra point is awarded if the solution does not declare any global variables.

Specific rules for Go solution

- Your source code must be a single file.
- Do not use any `for` and `goto` statements.
- Your solution will be tested against Go 1.24 (as of March 2025) or higher.

```
package main

func main() {
    ...
}
```

Specific rules for Python solution

- Your source code must be a single file, including at least a main function.
- Do not use any `for` loop, `while` loop, or any list / set / dictionary comprehension.
- Your solution will be tested against Python 3.13 (as of January 2025) or higher.

```
def main():
    ...

if __name__ == "__main__":
    main()
```

Sample Input

```
2
4
3 -1 1 10
5
9 -5 -5 -10 10
```

Sample Output

```
1
11250
```

Mission 2: Publish your source code as a secret gist

Description

To ensure confidentiality, we ask that you publish your source code as a secret gist on GitHub. This will allow us to review your work while keeping it private.

If you're not familiar with secret gists, no worries! We've got you covered. Simply follow the [step-by-step instructions in this helpful guide](#).

We will take a look at your source code later, after you achieve Mission 3.

Your program will be **auto-tested**, so please be **strict** about the input/output specification. For failed solutions, we will still assess it on a case-by-case but it will cause reduction to your overall score.

For the submission, we only accept a single file. The file must be named `main` with the proper file extension: `main.py` for Python or `main.go` for Go. Your source code will be evaluated upon completion of Mission 3.

Important reminder: Please avoid publishing your code as a public gist or deploying to any public-facing URLs. Make sure to upload only a single `main.py` or `main.go` file. Do not submit any additional files.

Mission 3: Send us the URL of your work

Description

First, construct a JSON string like below:

Go solution

```
{
  "github_url": "https://gist.github.com/YOUR_ACCOUNT/GIST_ID",
  "contact_email": "EMAIL",
  "solution_language": "golang"
}
```

Python solution

```
{
  "github_url": "https://gist.github.com/YOUR_ACCOUNT/GIST_ID",
  "contact_email": "EMAIL",
  "solution_language": "python"
}
```

Fill in your email address for `EMAIL`, and the path to your secret gist for `YOUR_ACCOUNT/GIST_ID`. In addition, `solution_language` should be filled with the solution language of your choice (`golang` or `python`). Be sure you have double-checked your email address; we will contact you by email.

Then, make an HTTP POST request to the following URL with the JSON string as the body part.

```
https://api.challenge.henнге.com/challenges/backend-recursion/004
```

Content-Type

The `Content-Type` of the request must be `application/json`.

Authorization

The URL is protected by HTTP Basic Authentication, which is explained on Chapter 2 of RFC2617, so you have to provide an `Authorization` header field in your POST request

- For the `userid` of HTTP Basic Authentication, use the same email address you put in the JSON string.
- For the `password`, provide a 10-digit time-based one time password conforming to RFC6238 TOTP.

Authorization password

For generating the TOTP password, you will need to use the following setup:

- You have to read RFC6238 (and the errata too!) and get a correct one time password by yourself.
- TOTP's Time Step X is 30 seconds, $T0$ is 0.
- Use `HMAC-SHA-512` for the hash function, instead of the default `HMAC-SHA-1`.
- Token shared secret is the userid followed by ASCII string value `"HENNGECHALLENGE004"` (not including double quotations).

Shared secret examples

- For example, if the userid is `"ninja@example.com"`, the token shared secret is `"ninja@example.comHENNGECHALLENGE004"`.
- For example, if the userid is `"nijasamuraisumotorishogun@example.com"`, the token shared secret is `"nijasamuraisumotorishogun@example.comHENNGECHALLENGE004"`

If your POST request succeeds, the server returns HTTP status code `200`.

Rules

- You do not have to disclose how you achieved this mission at this time. Do not hesitate to use source codes or tools on the net, but do the exploring process by yourself of course, do not ask your friend to help you.
- No bruteforce attacks, please!

Sample Request

```
POST /challenges/backend-recursion/004 HTTP/1.1
Authorization: Basic bmluanFAZXhhbXBsZS5jb206MTU5NTk0MjU2NA==
Host: api.challenge.henнге.com
Accept: */*
Content-Type: application/json
Content-Length: 133

{"github_url":"https://gist.github.com/henнге/b859bd12e7a7fb418141","contact_email":"ninja@example.com","solution_language":"golang"}
```

Sample Response

```
HTTP/1.1 200 OK
Content-Type: application/json
Date: Wed, 28 Jun 2019 02:15:16 GMT
Transfer-Encoding: chunked

{"message":"Congratulations! You have achieved mission 3"}
```

Any questions? Found a bug?

The challenge is designed to be completed independently. **To ensure a fair evaluation, please avoid contacting us with questions about the challenge itself.** However, if you encounter any internal server errors or 500 status codes, please do contact us for assistance. **When reporting such errors, please include screenshots, network logs, and/or a minimum reproducible example (steps to reproduce the error) to help us with the investigation.**

If you have any other questions or need assistance with something unrelated to the challenge, please don't hesitate to reach out to us at q10@henнге.com (for internship candidates) or recruit-engineer@henнге.com (for mid-career candidates).

Copyright

Do not share this challenge.

© HENNGE K.K. All rights reserved.

This coding challenge and all associated materials are the intellectual property of HENNGE K.K. Unauthorized reproduction, distribution or disclosure of any part of this content, in whole or in part, is strictly prohibited and constitutes intellectual property infringement.

HENNGE © 2025 | Contact us [here](#) | [Privacy Policy](#)

This coding challenge and all associated materials are the intellectual property of HENNGE K.K. Unauthorized reproduction, distribution or disclosure of any part of this content, in whole or in part, is strictly prohibited and constitutes intellectual property infringement.